

CESAR SCHOOL · ADS033 · 2026.2

MAESTRO

E2E WEB

Guia do estudante · Testes de ponta a ponta contra o SUT didático

DISCIPLINA	Testes Automatizados (ADS033) · 2026.2
SUT DE REFERÊNCIA	Cidade Conectada · SUT didático mínimo complementar
FERRAMENTA	Maestro CLI · driver Web (Beta)
PONTO DE PARTIDA	SUT didático rodando em <code>http://localhost:3000</code>
O QUE VOCÊ CONSTRÓI	Workspace <code>tests/e2e/maestro/</code> com 5 flows e 1 subflow reaproveitável
ENTREGA VINCULADA	Suíte e2e do Laboratório da TAS · Unidade 1

NESTE GUIA

SEIS ETAPAS

1 CONFIGURANDO O AMBIENTE³

- 1.1 Instalar a CLI do Maestro³
- 1.2 Subir o SUT³
- 1.3 Usuários semeados⁴
- 1.4 Estrutura de pastas do workspace de testes⁴
- 1.5 Arquivo de configuração do workspace (config.yaml)⁵

2 CRIANDO O PRIMEIRO FLOW⁶

- 2.1 Cuidado com o viewport pequeno no Maestro Web⁶

3 DEFININDO PASSOS REAPROVEITÁVEIS⁸

- 3.1 Login como subflow parametrizado⁸
- 3.2 Reset de estado como passo reaproveitável (via JavaScript)⁹
- 3.3 Subflows inline (para lógica curta)¹⁰

4 EXECUTANDO UM TESTE ÚNICO¹¹

5 MONTANDO UMA SUÍTE DE TESTES¹²

- 5.1 Executando a pasta inteira¹²
- 5.2 Filtrando por tags¹²
- 5.3 Ordem de execução e relatórios¹²
- 5.4 Organizando por subpastas¹³

6 INTEGRANDO COM O REPOSITÓRIO E CI¹⁴

- 6.1 Scripts de conveniência¹⁴
- 6.2 Exemplo de pipeline (GitHub Actions)¹⁴

BOAS PRÁTICAS¹⁵

1 CONFIGURANDO O AMBIENTE

CLI · SUT · USUÁRIOS SEMEADOS · WORKSPACE

O Maestro é um framework open source de automação de UI, originalmente focado em mobile (Android/iOS/React Native/Flutter), que hoje também suporta testes de aplicações web em navegadores Chromium (recurso em Beta). Os testes são escritos em Flows YAML declarativos, sem necessidade de instalar dependências no código da aplicação.

Este guia usa como alvo o **SUT didático Cidade Conectada** (sut-didatico/): uma API + UI web em Node.js puro, sem dependências externas, com usuários semeados e um endpoint exclusivo de teste para resetar o estado.

1.1 INSTALAR A CLI DO MAESTRO

terminal

```
# macOS / Linux (no Windows, use WSL2 e o mesmo comando)
curl -fsSL "https://get.maestro.mobile.dev" | bash
maestro --version

# Se o comando nao for reconhecido, o PATH nao foi recarregado:
export PATH="$PATH:$HOME/.maestro/bin"
```

O MAESTRO NÃO INSTALA NADA NO SUT

O Maestro roda como binário externo e interage com a aplicação da mesma forma que um usuário real, via camada de acessibilidade/DOM renderizado. Não é necessário instalar nada dentro de sut-didatico/.

1.2 SUBIR O SUT

Clone o SUT **ao lado** do seu projeto de testes, exatamente como o README.md dele recomenda:

estrutura do projeto

```
cidade-conectada/
├── tas-cidade-conectada/  <- sua suite de testes (inclui tests/e2e/maestro/)
└── sut-didatico/          <- alvo dos exemplos; somente leitura
```

terminal

```
cd sut-didatico
npm start                # http://localhost:3000
# PORT=3100 npm start    # se a 3000 estiver ocupada
```

Recurso	Endereço
API	http://localhost:3000/api/v1
Interface web	http://localhost:3000/web

Reset do estado	POST /api/v1/_test/reset
-----------------	--------------------------

Os Flows deste guia testam a **interface web** (/web). O endpoint de reset é o que torna os testes independentes de execuções anteriores — voltamos a ele na seção 3.2.

1.3 USUÁRIOS SEMEADOS

O SUT já nasce com estes usuários, usados diretamente nos Flows sem precisar de tela de cadastro:

E-mail	Perfil	Senha
ana@exemplo.com	CITIZEN	senha123
bruno@exemplo.com	CITIZEN	senha123
gestor@exemplo.com	MANAGER	senha123

A UI WEB SÓ TEM TELAS DE CIDADÃO

A UI web só implementa as telas de **cidadão** (login, nova demanda, minhas demandas). Não há tela de gestor — transição de status, histórico e listagem paginada existem na API, mas fazem mais sentido como testes de serviço do que de UI neste SUT.

1.4 ESTRUTURA DE PASTAS DO WORKSPACE DE TESTES

O workspace do Maestro fica em `tests/e2e/maestro/`, dentro do repositório de testes (`tas-cidade-conectada/`):

estrutura do workspace

```
tas-cidade-conectada/
├── tests/
│   └── e2e/
│       └── maestro/
│           ├── config.yaml          # configuracao da suite
│           ├── flows/              # flows de topo (casos de teste)
│           │   ├── login_citizen_sucesso.yaml
│           │   ├── login_credenciais_invalidas.yaml
│           │   ├── criar_demanda_sucesso.yaml
│           │   ├── criar_demanda_descricao_curta.yaml
│           │   └── listar_demandas_atualiza_apos_criacao.yaml
│           ├── subflows/           # passos reaproveitaveis
│           │   └── login.yaml
│           ├── scripts/            # JavaScript auxiliar (ex.: reset via API)
│           │   └── resetar_estado.js
```

1.5 ARQUIVO DE CONFIGURAÇÃO DO WORKSPACE (CONFIG.YAML)

tests/e2e/maestro/config.yaml

```
flows:
  - "flows/*"          # apenas os flows de topo entram na suite

executionOrder:
  continueOnFailure: true # roda tudo mesmo se um flow falhar, e reporta o resultado de cada um
```

2 CRIANDO O PRIMEIRO FLOW

SELETORES POR ID · VIEWPORT PEQUENO

Um Flow web usa url (em vez de appId, usado em mobile) no cabeçalho. O `index.html` do SUT expõe os elementos com atributo `id` (que coincide com `data-testid`), o que torna os seletores estáveis mesmo se o texto da tela mudar:

`tests/e2e/maestro/flows/login_citizen_sucesso.yaml`

```
url: http://localhost:3000/web
tags:
  - smoke
  - login
---
- launchApp
- tapOn:
  id: "campo-email"
- inputText: "ana@exemplo.com"
- tapOn:
  id: "campo-senha"
- inputText: "senha123"
- tapOn:
  id: "botao-entrar"
- assertVisible: "Autenticado como Ana Cidada"
- scrollUntilVisible:
  element:
    id: "campo-descricao"
```

Comandos usados: `launchApp`, `tapOn`, `inputText`, `assertVisible`, `scrollUntilVisible`. O Maestro prioriza texto visível ao usuário para localizar elementos, mas para campos de formulário é mais robusto selecionar por `id` — evita que o teste quebre se o placeholder ou rótulo mudar.

2.1 CUIDADO COM O VIEWPORT PEQUENO NO MAESTRO WEB

O Maestro Web roda num viewport baixo e fixo nesta versão Beta (por volta de **500px de altura** — visto no log de execução como `heightPixels=502`). Boa parte de uma página comum fica **abaixo da dobra** assim que o conteúdo cresce, como acontece aqui depois do login.

`assertVisible` **não rola a página** — ele só confirma o que já está na tela e falha com `Assertion 'id: campo-descricao is visible' failed` se o elemento existir no DOM mas estiver fora do viewport. Para elementos abaixo da dobra, use `scrollUntilVisible` antes de um `tapOn` ou de outro `assertVisible`:

trecho de flow

```
- scrollUntilVisible:
  element:
    id: "campo-descricao"
- tapOn:
  id: "campo-descricao"
```

Regra prática usada em todos os flows deste guia: qualquer elemento que não esteja na primeira "tela" da aplicação leva um `scrollUntilVisible` antes de ser tocado ou verificado.

OUTRO DETALHE DESTE SUT

`login_citizen_sucesso.yaml` verifica apenas `campo-descricao`, não `lista-demandas`. Logo após o login, antes de qualquer demanda ser criada, a `<ul>` da lista está vazia — sem `<li>` dentro, ela não tem altura

suficiente para o `scrollUntilVisible` conseguir localizá-la.

Quem cobre a lista de fato é `listar_demandas_atualiza_apos_criacao.yaml`, que só a verifica depois de criar uma demanda, quando ela já tem conteúdo.

Dica para descobrir seletores: como o `index.html` deste SUT é HTML simples sem framework, o jeito mais direto é abrir `http://localhost:3000/web` num navegador comum e usar o DevTools (F12 → inspecionar elemento) para ver o `id/data-testid` de cada elemento.

3 DEFININDO PASSOS REAPROVEITÁVEIS

SUBFLOWS · RUNFLOW · RESET VIA JAVASCRIPT

3.1 LOGIN COMO SUBFLOW PARAMETRIZADO

O login se repete em praticamente todo Flow (criar demanda, listar demandas etc.). Em vez de duplicar os passos, extraímos um subflow que recebe EMAIL/SENHA como parâmetros:

tests/e2e/maestro/subflows/login.yaml

```
url: http://localhost:3000/web
---
- tapOn:
  id: "campo-email"
- inputText: ${EMAIL}
- tapOn:
  id: "campo-senha"
- inputText: ${SENHA}
- tapOn:
  id: "botao-entrar"
```

TODO .YAML PRECISA DE UMA CONFIG SECTION

O Maestro exige uma "Config Section" de verdade (não apenas comentários) acima do --- em todo arquivo .yaml, inclusive subflows chamados só via runFlow. Um cabeçalho só com comentários falha com Config Section Required.

Por isso o subflow repete o url: do flow que o chama, mesmo já herdando o contexto em tempo de execução.

O subflow **não faz asserções** — cada Flow que o chama decide o que verificar (login válido, credenciais inválidas etc.), o que o mantém útil tanto para caminhos felizes quanto negativos. Chamando o subflow com runFlow + env:

tests/e2e/maestro/flows/criar_demanda_sucesso.yaml

```
url: http://localhost:3000/web
tags:
  - smoke
  - demanda
---
- launchApp
- runFlow:
  file: ../subflows/login.yaml
  env:
    EMAIL: ana@exemplo.com
    SENHA: senha123
- scrollUntilVisible:
  element:
    id: "campo-descricao"
- tapOn:
  id: "campo-descricao"
- inputText: "Buraco grande na pista sentido bairro, proximo ao numero 450."
- tapOn:
  id: "botao-enviar"
- assertVisible: "Demanda registrada."
- assertVisible: "Protocolo DEM-.*"
```

SELECTS FICAM NO VALOR PADRÃO DE PROPÓSITO

Os campos "Categoria" e "Região" são <select> HTML com valores padrão já válidos (ROAD_MAINTENANCE / RPA_3), então este Flow nem precisa interagir com eles — foca no campo de descrição, que é o dado que varia por caso de teste.

Trocar a opção selecionada de um <select> nativo é mais frágil de automatizar; se for necessário, vale isolar isso em um Flow próprio.

3.2 RESET DE ESTADO COMO PASSO REAPROVEITÁVEL (VIA JAVASCRIPT)

O SUT expõe POST /api/v1/_test/reset só para laboratório: zera demandas e re-semeia os usuários. Isso torna cada Flow independente da ordem de execução — sem ele, um Flow que lista demandas dependeria de quantas demandas os Flows anteriores criaram.

O Maestro permite chamar APIs HTTP via JavaScript embutido:

```
tests/e2e/maestro/scripts/resetar_estado.js
```

```
http.post('http://localhost:3000/api/v1/_test/reset', { body: '' });
```

HTTP.POST EXIGE BODY, MESMO VAZIO

http.post do Maestro exige um body, mesmo que vazio. Chamar http.post(url) sem esse parâmetro falha com CommandFailed: method POST must have a request body.

E o script roda automaticamente no início de cada Flow com o hook onFlowStart:

```
tests/e2e/maestro/flows/listar_demandas_atualiza_apos_criacao.yaml
```

```
url: http://localhost:3000/web
tags:
  - demanda
onFlowStart:
  - runScript: ../scripts/resetar_estado.js
  ---
  - launchApp
  - runFlow:
      file: ../subflows/login.yaml
      env:
        EMAIL: bruno@exemplo.com
        SENHA: senha123
  - scrollUntilVisible:
      element:
        id: "campo-descricao"
  - tapOn:
      id: "campo-descricao"
  - inputText: "Lixo acumulado na esquina da Rua Aurora ha varios dias."
  - tapOn:
      id: "botao-enviar"
  - scrollUntilVisible:
      element:
        id: "botao-atualizar"
  - tapOn:
      id: "botao-atualizar"
  - scrollUntilVisible:
      element:
        id: "lista-demandas"
  - assertVisible: ".*DEM-.*"
```

Esse mesmo `onFlowStart` deve ser adicionado a **todo** Flow que dependa de estado limpo — todos, neste SUT.

3.3 SUBFLOWS INLINE (PARA LÓGICA CURTA)

Quando não vale a pena criar um arquivo separado, use `commands` inline com um `label` descritivo. Exemplo hipotético, caso a UI ganhasse um banner de cookies:

trecho de flow

```
- runFlow:
  label: "Fechar banner de cookies, se existir"
  commands:
    - tapOn:
      text: "Aceitar cookies"
      optional: true
```

Organize os subflows por responsabilidade (`subflows/login.yaml`, `subflows/demanda.yaml` se o projeto crescer) — assim, se a UI de login mudar, você ajusta um único arquivo e todos os Flows que o utilizam continuam funcionando.

4 EXECUTANDO UM TESTE ÚNICO

MAESTRO TEST · MODO CONTÍNUO

Com o SUT rodando em outro terminal (npm start dentro de sut-didatico/):

```
terminal
```

```
maestro test tests/e2e/maestro/flows/login_citizen_sucesso.yaml
```

Outras opções úteis:

```
terminal
```

```
# Modo contínuo: reexecuta o flow a cada alteracao salva no YAML
maestro test -c tests/e2e/maestro/flows/login_citizen_sucesso.yaml

# -e sobrescreve ${VAR} referenciadas no flow. login_citizen_sucesso.yaml
# NAO usa ${EMAIL} (chama o subflow com valores literais), entao -e nao
# tem efeito nele. Quem le ${EMAIL}/${SENHA} e o subflow:
maestro test -e EMAIL=bruno@exemplo.com -e SENHA=senha123 tests/e2e/maestro/subflows/login.yaml
```

Um bom primeiro teste para validar a instalação: rode login_citizen_sucesso.yaml e confirme que a asserção "Autenticado como Ana Cidada" passa — isso comprova que o Chromium gerenciado pelo Maestro, o SUT e o endpoint de reset estão todos se comunicando corretamente.

5 MONTANDO UMA SUÍTE DE TESTES

TAGS · ORDEM DE EXECUÇÃO · RELATÓRIOS

5.1 EXECUTANDO A PASTA INTEIRA

terminal

```
maestro test tests/e2e/maestro/
```

Isso roda os cinco Flows do workspace: dois de login (sucesso e credenciais inválidas), dois de criação de demanda (sucesso e descrição curta) e um de listagem. O comando só é bem-sucedido se **todos** passarem.

5.2 FILTRANDO POR TAGS

Cada Flow foi marcado com tags por natureza do caso:

Flow	Tags
login_citizen_sucesso	smoke, login
login_credenciais_invalidas	login, negativo
criar_demanda_sucesso	smoke, demanda
criar_demanda_descricao_curta	demanda, negativo
listar_demandas_atualiza_apos_criacao	demanda

terminal

```
# so os smoke tests (rapido, para todo PR)
maestro test tests/e2e/maestro/ --include-tags=smoke

# tudo, exceto os casos negativos
maestro test tests/e2e/maestro/ --exclude-tags=negativo
```

5.3 ORDEM DE EXECUÇÃO E RELATÓRIOS

config.yaml já define `continueOnFailure: true`, então um Flow quebrado (por exemplo, se alguém aplicar a alteração didática `status: 'RECEIVED' → 'NEW'` sugerida no README.md do SUT) não interrompe a execução dos demais — todos rodam e o relatório mostra exatamente qual falhou.

Gerar relatório da suíte (JUnit ou HTML, útil para CI):

terminal

```
maestro test tests/e2e/maestro/ --format junit --output maestro-report.xml
maestro test tests/e2e/maestro/ --format html --output maestro-report.html
```

5.4 ORGANIZANDO POR SUBPASTAS (SE A SUÍTE CRESCER)

Se o número de Flows aumentar e fizer sentido agrupar por funcionalidade (flows/login/, flows/demandas/), declare os padrões de inclusão em config.yaml:

tests/e2e/maestro/config.yaml

```
flows:
  - "flows/login/*"
  - "flows/demandas/*"
  # ou, recursivamente:
  # - "flows/**"
```

6 INTEGRANDO COM O REPOSITÓRIO E CI

GITHUB ACTIONS · SCRIPTS DE CONVENIÊNCIA

6.1 SCRIPTS DE CONVENIÊNCIA

Como o SUT já define "start": "node server.js" no package.json, um pipeline típico é: subir o SUT em background, esperar ele responder, rodar a suíte.

6.2 EXEMPLO DE PIPELINE (GITHUB ACTIONS)

`.github/workflows/e2e-maestro.yml`

```
name: e2e-maestro
on: [push, pull_request]
jobs:
  e2e-web:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          # checkout do repo que contem tas-cidade-conectada/ e sut-didatico/ lado a lado
          fetch-depth: 0
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
      - name: Instalar Maestro
        run: |
          curl -fsSL "https://get.maestro.mobile.dev" | bash
          echo "$HOME/.maestro/bin" >> "$GITHUB_PATH"
      - name: Subir o SUT didatico em background
        working-directory: sut-didatico
        run: npm start &
      - name: Aguardar o SUT responder
        run: npx wait-on http://localhost:3000/api/v1/auth/me --http-timeout 2000 || true
      - name: Rodar a suite Maestro
        working-directory: tas-cidade-conectada
        run: maestro test tests/e2e/maestro/ --format junit --output maestro-report.xml
      - name: Publicar relatorio
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: maestro-report
          path: tas-cidade-conectada/maestro-report.xml
```

BOAS PRÁTICAS

POR QUE O WORKSPACE É DESENHADO ASSIM

Prática	Por que
Estado limpo por Flow	onFlowStart + resetar_estado.js elimina dependência de ordem de execução — cada Flow parte de zero demandas e dos três usuários semeados.
Seletores por id, não por texto	Como o index.html já expõe id/data-testid coincidentes, os Flows usam id: "campo-descricao" em vez de depender de placeholder ou rótulo, que são mais fáceis de mudar sem querer.
Subflow sem asserções	subflows/login.yaml só executa a ação; quem decide o que verificar é o Flow chamador — permite reusar o mesmo subflow em casos positivos e negativos.
Tags por natureza do caso	smoke, negativo, área funcional — permitem rodar um subconjunto rápido em PRs e a suíte completa antes de merge.
Escopo consciente	A suíte cobre só o que a UI web realmente expõe (login e ações de cidadão). Fluxos de gestor não têm tela correspondente neste SUT e são mais adequados a testes de API/serviço.
scrollUntilVisible antes de elementos abaixo da dobra	O viewport do Maestro Web é baixo (~500px) e assertVisible não rola a página — sem isso, qualquer elemento fora da primeira tela falha, mesmo existindo no DOM.

O SUPORTE WEB DO MAESTRO AINDA É BETA

Cobre bem cenários Chromium, mas para casos muito específicos de navegador (múltiplas abas, extensões etc.) avalie complementar com Playwright ou Cypress.